

系統設計面試 · 第八章

設計短網址服務

Design a URL Shortener

像 TinyURL 一樣：把冗長的網址變成短小、可分享的連結，並支撐每天上億次的建立與重導向請求。

依據《System Design Interview》Ch.8 內容整理



面試四步驟框架

本章依循標準的系統設計面試流程展開



STEP 1

釐清問題、確立範圍

提出澄清式問題，對齊需求與規模假設



STEP 2

提出高階設計

API 端點、重導向與縮短流程，取得面試官認同



STEP 3

深入設計

資料模型、雜湊函式、縮短與重導向的細節



STEP 4

總結與延伸

限流、擴展、分析等加分議題

Step 1| 釐清問題、確立範圍

透過澄清式問題，把開放題收斂成明確需求

向面試官確認的重點

運作方式 長網址 → 產生較短的別名，點擊後導回原網址

流量規模 每天產生 1 億筆短網址

短網址長度 越短越好

允許字元 0-9、a-z、A-Z 共 62 種

刪除 / 更新 簡化假設：不支援

三大核心需求



網址縮短

輸入長網址，回傳短網址



網址重導向

輸入短網址，導回原始長網址



非功能需求

高可用、可擴展、容錯

Step 1| 封底估算 (Back-of-the-Envelope)

與面試官一起走過假設與計算，確保雙方在同一頁

1,160 / 秒

寫入 QPS $1 \text{ 億} \div 24 \div 3600$

11,600 / 秒

讀取 QPS 假設讀寫比 10 : 1

3,650 億筆

十年總記錄數 $1 \text{ 億} \times 365 \text{ 天} \times 10 \text{ 年}$

36.5 TB

十年儲存需求 平均網址長度 100 bytes

Step 2|API 設計

採用 REST 風格，兩個端點即可支撐核心功能

POST

建立短網址

POST `api/v1/data/shorten`

→ 請求參數 : { longUrl: longURLString }

→ 回傳 : shortURL

GET

網址重導向

GET `api/v1/shortUrl`

→ 回傳 : 對應的 longURL

→ 由伺服器執行 HTTP 重導向

Step 2|301 vs 302 重導向

兩種 HTTP 狀態碼，對應不同的營運考量

	301 永久重導向	302 暫時重導向
語意	網址「永久」搬移到長網址	網址「暫時」搬移到長網址
瀏覽器行為	快取回應；後續請求直接到長網址伺服器	每次請求都先經過短網址服務
適用情境	想降低伺服器負載	重視點擊率、來源等分析數據



實作上最直觀的做法：以雜湊表存 `<shortURL, longURL>`，查到 `longURL` 後執行重導向。

Step 3 | 資料模型與短網址長度

從記憶體雜湊表走向關聯式資料庫



資料模型

全部塞進記憶體雜湊表僅是起點——記憶體昂貴且有限，實務上應存入關聯式資料庫。

url 資料表	
id	(PK)
shortURL	
longURL	



短網址要幾個字元？

字元集 [0-9, a-z, A-Z] 共 62 種，需找最小的 n 使 $62^n \geq 3,650$ 億。

$$n = 7$$

$62^7 \approx 3.5$ 兆，足以涵蓋十年 3,650 億筆的需求，因此短網址長度取 7 個字元。

Step 3| 兩種雜湊策略比較

「雜湊 + 碰撞處理」 vs 「Base 62 轉換」

	雜湊 + 碰撞處理	Base 62 轉換
做法	以 CRC32 / MD5 / SHA-1 雜湊後取前 7 碼；碰撞時附加預定字串重試（可用布隆過濾器加速查重）	先由唯一 ID 產生器取得數字 ID, 再轉成 62 進位字串
長度	固定 7 碼	不固定，隨 ID 增大
相依性	不需唯一 ID 產生器	依賴分散式唯一 ID 產生器
可預測性	無法從舊網址推測新網址	ID 遞增，下一個短網址可被猜到，有安全疑慮



本章設計採用 Base 62 轉換：流程簡單、無碰撞問題。範例： $11157_{10} \rightarrow [2, 55, 59] \rightarrow "2TX"$

Step 3 | 縮短流程深入設計

以 Base 62 為核心的寫入路徑

- 1 輸入 longURL
- 2 查資料庫：是否已存在？
- 3 已存在 → 直接回傳既有 shortURL
- 4 不存在 → 唯一 ID 產生器配發新 ID(主鍵)
- 5 以 Base 62 將 ID 轉為 shortURL
- 6 寫入 id、shortURL、longURL, 回傳結果

實際範例

longURL	en.wikipedia.org/wiki/ Systems_design
產生的 ID	2009215674938
Base 62	zn9edcu
短網址	tinyurl.com/zn9edcu

Step 3 | 重導向流程深入設計

讀多寫少 → 以快取為先的讀取路徑



- ✓ 讀寫比 10:1 → <shortURL, longURL> 映射放進快取, 加速查詢
- ✓ 快取命中: 直接回傳 longURL; 未命中: 查資料庫後回填
- ✓ 資料庫也查不到 → 很可能是使用者輸入了無效的短網址

Step 4 | 總結與延伸議題

核心設計已完成 :API、資料模型、雜湊函式、縮短與重導向。若面試還有時間，可加碼以下 talking points:



限流器 (Rate Limiter)

過濾惡意的大量縮短請求，可依 IP 等規則過濾



Web 層擴展

Web 層無狀態，可隨時增減伺服器水平擴展



資料庫擴展

複寫 (replication) 與分片 (sharding) 是常見手段



數據分析

整合分析方案，回答「誰在何時點了連結」等商業問題



可用性 / 一致性 / 可靠性

大型系統成功的核心，任何設計都應回頭檢視



回顧相關章節

唯一 ID 產生器 (Ch.7)、限流器 (Ch.4)、系統基礎 (Ch.1)